

ネットワーク実習: TCPスライディングウィンドウの観察(準備運動編)

1. はじめに: 今日の準備運動で学ぶこと

皆さん、本番のルーター演習に入る前に、まずは軽い「準備運動」をしましょう！

テーマは「パケットの旅をあえて遅らせて、TCPの知恵を覗いてみよう」です。

私たちがWebページを見たりファイルをダウンロードしたりするとき、裏側では **TCP (Transmission Control Protocol)** というプロトコルが使われています。TCPの最大の使命は、データを「信頼性高く、かつできるだけ高速に」届けることです。

もし、パケットを1個送るたびに「届いたよ！」という返事(ACK: 確認応答)を待ってから次のパケットを送っていたらどうなるでしょう？日本とアメリカのように距離が離れていて往復に時間がかかる(遅延が大きい)回線では、通信速度がカタツムリのように遅くなってしまいます。

そこでTCPは、「相手の受け入れ許可(ウィンドウサイズ)の分だけ、返事を待たずに一気に送りつける」という超スマートな技を使っています。これが **スライディングウィンドウ (Sliding Window)** です。

この実習では、2台のPCの間にわざと「100ミリ秒」の遅延(タイムラグ)を発生させ、以下の2つのアプローチでTCPの「脳みそ」と「実際のパケットの流れ」を同時に観察します！

1. **ss コマンド**: 送信側PCの「内部パラメータ(ウィンドウサイズの変化)」をリアルタイムで覗き見る。
2. **tcpdump コマンド**: 回線上を流れる「生のパケット(ACKを待たずに一気に送られる様子)」をキャプチャする。

2. 実習環境(トポロジー図)

準備運動のネットワークは非常にシンプルです。2つの仮想マシン(Namespace)が、1本の仮想LANケーブル(vethペア)で直結されています。

```
[node1 (送信側)] ----- [node2 (受信側)]  
(192.168.1.1)      [vethペア]      (192.168.1.2)  
(veth-node1)      ※ここに意図的な遅延をかける      (veth-node2)
```

- **node1 (送信側)**: IPアドレス 192.168.1.1 / インターフェース veth-node1
- **node2 (受信側)**: IPアドレス 192.168.1.2 / インターフェース veth-node2

3. ステップ1: 環境の確認と、普通の通信

まずは何事も起きていない「超快適な回線」での疎通を確認します。

① node1からnode2へping(疎通確認)を打つ

ターミナルで以下のコマンドを実行し、2点間でパケットが何ミリ秒で往復しているか確認しましょう。

```
sudo ip netns exec node1 ping -c 3 192.168.1.2
```

【注目ポイント】

実行結果の最後の部分に `time=0.045 ms` のように表示されているはずです。

同じPCの中の仮想空間同士の通信なので、ほぼ「遅延ゼロ(0.1ミリ秒以下)」で光速のバケツリレーが行われています。

4. ステップ2: 遅延(ディレイ)の罠を仕掛ける！

さあ、ここからが実験です。この仮想ケーブルに、宇宙の距離(レイテンシ)を与えてみましょう。

Linuxには **tc**(**Traffic Control**) という、ネットワークの帯域や遅延をわざとコントロールできる悪魔のような(そして開発者にとっては神のような)コマンドがあります。

① ケーブルに「片道100ms」の遅延を仕込む

node1の送信ポート(veth-node1)に対して、パケットの送信を100ミリ秒(0.1秒)遅らせるルールを追加します。

```
sudo ip netns exec node1 tc qdisc add dev veth-node1 root netem delay 100ms
```

② pingで遅延を確認する

仕掛けが成功したか、もう一度 ping を打ってみましょう。

```
sudo ip netns exec node1 ping -c 3 192.168.1.2
```

【驚きの結果】

応答結果を見てください！ `time=100.xxx ms` のように、ピッタリ100ミリ秒遅れて返ってくるようになりました。

パケットが仮想ケーブルを渡るのに、わざと「100ミリ秒」の足止めを食らっている証拠です。

5. ステップ3: TCPスライディングウィンドウと生パケットの動きをモニターしよう！

パケットの往復に往復200ms近くかかるようになったこの「のろま回線」で、大量のデータを送るとTCPはどう動くでしょうか？

ここからは、3つ(または4つ)のターミナルを同時に使いこなして、多角的にモニターを行います！

【準備編】各ターミナルをスタンバイさせる

ターミナルA(受信側・node2)

受信側の node2 で、データを受け入れるための簡易サーバー(iperf3)を起動して待ち受けます。

```
sudo ip netns exec node2 iperf3 -s
```

(※このターミナルは、通信が終わるまでこのままにしておきます)

ターミナルB(観測:生パケットキャプチャ)

新規ターミナルを開き、tcpdump を使って、仮想ケーブル(veth-node1)を流れる生のTCPパケットをリアルタイム表示します。

```
sudo ip netns exec node1 tcpdump -i veth-node1 -n tcp -v
```

(※実行すると「パケット待ち」の静かな状態になります。通信が始まると、ここに激しくデータが流れます)

tcpdumpコマンドのオプション解説

- -i veth-node1: 観測するインターフェース(虫眼鏡を当てる場所)を指定。
- -n: IPアドレスやポート番号を名前解決(DNS検索)せず、数字のまま表示して高速化。
- tcp: TCPプロトコルのパケットだけをフィルタリングして表示。
- -v: 詳細情報(ウィンドウサイズやTCPオプション等)を表示。

ターミナルC(観測:送信側PC of 内部パラメータ)

もう一つ新規ターミナルを開き、送信側 node1 のTCP接続状況(ソケット情報)を1秒おきに監視します。

```
watch -n 1 "sudo ip netns exec node1 ss -t -i"
```

(※こちら最初は何も表示されませんが、送信が始まると接続情報が現れます)

【実験開始】データを送信し、観測せよ！

ターミナルD(送信実行)

最後のターミナルを開き、node1 から node2 へ向けて10秒間データを一気に流し込みます！

```
sudo ip netns exec node1 iperf3 -c 192.168.1.2 -t 10
```

6.【解説】モニター画面から読み解く「TCPの知恵」

データ送信中、各ターミナルで何が起きていたかをじっくり解説します。

① ターミナルC(ss -t -i)の観察ポイント

データ送信中、以下のような内部情報が1秒おきに表示されます。

```
ESTAB 0 128000 192.168.1.1:xxxxx 192.168.1.2:5201
cubic wscale:7,7 rto:300 rtt:100.12/0.05 ...
cwnd:45 ssthresh:32 ...
```

- **cwnd:45**(輻輳ウィンドウ)

TCPが「相手からの返事(ACK)を待たずに、一気に送り出してよいパケットの数」です。

最初は小さかったこの数値が、データの送信が進むにつれて「45(約45パケット＝約65KB分)」などと大きく広がっていく様子が見えましたか？

遅延が大きい回線でも、TCPは「お、パケットが詰まることなくちゃんとACKが返ってくるぞ。もっと一気に送り出そう！」と判断し、この cwnd(送り出す窓枠)を徐々にスライディングさせて広げていくのです。

② ターミナルB(tcpdump)の観察ポイント

パケットキャプチャ画面には、以下のような生々しい情報の行が超高速で流れます。

1) 最初の交渉(3ウェイ・ハンドシェイク時のウィンドウスケール確認)

```
192.168.1.1.5201 > 192.168.1.2.xxxxx: Flags [S], ... options [mss 1460,wscale 7]
```

- **options [wscale 7]** 通信を開始する一番最初のパケット(SYNパケット)で、「僕たちの受信能力を示すメモリ(ウィンドウサイズ)を、お互い 2^7 (128)倍にして広く使おうね！」というスケール交渉をしているのが分かります。

2) データ転送中(ACKを待たずに一気に送りつける瞬間)

```
12:00:00.100 192.168.1.1 > 192.168.1.2: Flags [.] , seq 1001:2460, length 1459
12:00:00.101 192.168.1.1 > 192.168.1.2: Flags [.] , seq 2461:3920, length 1459
12:00:00.102 192.168.1.1 > 192.168.1.2: Flags [.] , seq 3921:5380, length 1459
... (送信側がひたすら送り続ける) ...
12:00:00.300 192.168.1.2 > 192.168.1.1: Flags [.] , ack 1001, win 2048
```

- ここがスライディングウィンドウの 最も美しい瞬間 です！
- タイムスタンプ(左端の時刻)に注目してください。送信側(192.168.1.1)は、**0.3秒**に受信側から返ってくるはずの **ack 1001**(1001番まで無事に届いたよ！)という返事(**ACK**)を待つより前に、**seq 1001**、**seq 2461**、**seq 3921**...と、何個ものデータをノンストップで一気に相手に送りつけています。
- もしこれが「1個送って、返事を待つ」方式(ストップ＆ウェイト)だったら、100msの遅延があるため、1往復(RTT 200ms)に1パケットしか送れず、使い物にならない超極遅回線になっていたことでしょう。

7. 発展考察:なぜ「ウィンドウ」が必要なのか？(帯域遅延積のハナシ)

実習お疲れ様でした！

生パケットの旅を tcpdump で追いかけることで、TCPが「相手の受け入れ可能サイズ(win)」と「回線のゆとり(cwnd)」という2つのウィンドウを器用に使い分けながら、データを隙間なくパイプラインに流し込んでいる姿が見えたはずです。

ここで、ネットワーク設計における最重要物理法則「**BDP(Bandwidth-Delay Product: 帯域遅延積)**」をおさらいしましょう。

$$\text{BDP(ネットワークに溜め込めるデータ量)} = \text{帯域幅(通信速度)} \times \text{往復遅延(RTT)}$$

もし、あなたが「1Gbps」という超高速な光回線を持っていても、遅延(RTT)が「100ms」あれば、TCPの ウィンドウサイズは最低でも以下のサイズまで広がらないと、回線のポテンシャルを100%引き出すことはできません。

$$\text{BDP} = 1 \text{ Gbps} \times 0.1 \text{ 秒} = 100 \text{ メガビット} \approx 12.5 \text{ メガバイト}$$

つまり、「**12.5MB**分ものデータを、返事を一切待たずに一気に送り続けられる巨大なウィンドウ(バッファ)」がルーターやPCに備わっていないと、いくら回線が太くても宝の持ち腐れになってしまうのです。

もし実験が終わって、遅延設定を元に戻したいときは、以下のコマンドを実行して悪魔の仕掛けを解除してください。

```
# 遅延設定の削除(元の快適な回線に戻す)
sudo ip netns exec node1 tc qdisc del dev veth-node1 root
```

これで準備運動はバッチリです！

パケットを「どう送るか(TCP)」の賢さを tcpdump と ss で学んだ皆さんなら、次は「どの道を通すか(ルーティング)」というルーターたちの戦略がより深く理解できるはずです。それでは、本番の三角形ネットワーク実習に進みましょう！